

# 1 - Exam facts, and how to attack a question

Read this page first on exam day. It is worth more than any single fact.

|                                  |                                      |                                      |                                                            |
|----------------------------------|--------------------------------------|--------------------------------------|------------------------------------------------------------|
| <b>Questions</b><br>60           | <b>Time</b><br>120 min (2 min each)  | <b>Pass mark</b><br>720 / 1000       | <b>Scenarios</b><br>4, drawn from 6                        |
| <b>D1 Agentic</b><br>27% (~16 q) | <b>D2 Tools + MCP</b><br>18% (~11 q) | <b>D3 Claude Code</b><br>20% (~12 q) | <b>D4 Prompts / D5 Context</b><br>20% (~12 q) / 15% (~9 q) |

Each question states **how many answers to select** - read that line. No penalty for guessing, so never leave one blank. If a question takes more than 3 minutes, mark it and come back.

## The 30-second method

|   |                                                                                               |
|---|-----------------------------------------------------------------------------------------------|
| 1 | Read the <b>last sentence</b> of the question first. That is what is actually asked.          |
| 2 | Note the signal words (page 2). They usually name the answer type.                            |
| 3 | Remove any answer describing a feature that <b>does not exist</b> .                           |
| 4 | Remove any answer blaming a part the question says is <b>working correctly</b> .              |
| 5 | From what is left, choose the <b>weakest fix that still meets the requirement</b> .           |
| 6 | Ask <b>where the information should have been PRODUCED</b> , not where the failure showed up. |

**Step 6 - your one cross-domain weakness.** The failure appears late; the fix belongs early. Field missing that the source always has → **required**, not nullable-plus-a-null-check. Inconsistent dates and currency → **normalisation rules in the prompt**, not a post-processing layer. Synthesis cannot judge a source → **subagents record dates, location and methodology**, not re-fetching later. The late fix is defensible engineering - that is why it is the distractor. Which component held the information when it was lost?

## The strength ladder - choose the lowest level that meets the requirement

| Level | Fix                                                                              | Use when                                                      |
|-------|----------------------------------------------------------------------------------|---------------------------------------------------------------|
| 1     | Better <b>tool descriptions</b>                                                  | Model picks the wrong tool; descriptions are short or similar |
| 2     | <b>Few-shot examples</b> (2-4)                                                   | Behaviour is inconsistent in unclear cases                    |
| 3     | <b>Explicit criteria</b> in the prompt                                           | Too many false alarms; current instruction is vague           |
| 4     | <b>Programmatic enforcement</b><br>hooks, prerequisite gates, forced tool_choice | The rule must ALWAYS work                                     |
| 5     | Separate classifier / ML system                                                  | Almost never the answer on this exam                          |

|                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Level 4 wins whenever the question mentions any of these</b>                                                                      |
| Money (refunds, payments, billing) - identity checks - policy limits or thresholds                                                   |
| The words <b>deterministic, guaranteed, must, or never</b>                                                                           |
| Reason from the guide: prompt instructions have a <b>non-zero failure rate</b> . Sometimes is not acceptable when money is involved. |

## The five kinds of wrong answer

|                               |                                                                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Too big a solution</b>     | Builds new systems before trying simple fixes. Signals: train a classifier, deploy a model, add a routing layer, build a service |
| <b>Blames a working part</b>  | The question names the broken part. Any answer pointing elsewhere is wrong                                                       |
| <b>Feature does not exist</b> | Sounds confident, describes a flag or file that is not real (see page 2)                                                         |
| <b>Solves another problem</b> | Reasonable, but fixes something the question did not ask about                                                                   |
| <b>Moves work to people</b>   | Asks developers or customers to do more, or hides real problems                                                                  |

## Never the right answer

|                                                                                       |
|---------------------------------------------------------------------------------------|
| Self-reported confidence used to decide when to <b>escalate</b>                       |
| <b>Sentiment analysis</b> used to judge how difficult a case is                       |
| A <b>bigger context window</b> used to fix inconsistent quality or attention dilution |
| <b>Voting across several runs</b> to filter findings - this hides real bugs           |
| Reading the model's <b>text</b> to decide when the loop stops - use stop_reason       |
| A fixed <b>iteration cap</b> as the main stopping method (fine as a safety limit)     |
| <b>Hiding errors</b> by returning empty results as success                            |
| <b>Stopping the whole workflow</b> because one subagent failed                        |
| <b>Guessing</b> when a tool returns several matches - ask for another identifier      |

## 2 - Signal words, paths, and flags

Domain 3 is 20% of the exam and almost entirely memorisation. These are the cheapest points.

### Signal words in the question

| Words in the question                        | What to choose                                    |
|----------------------------------------------|---------------------------------------------------|
| most effective <b>first step</b>             | The cheap simple fix (level 1-3), not the big one |
| deterministic, guaranteed, must never        | Programmatic enforcement (level 4)                |
| the logs show the coordinator...             | The cause is named. Fix that part                 |
| minimal descriptions                         | Improve the tool descriptions                     |
| blocking, developers wait                    | Real-time API, not batch                          |
| overnight, next morning, weekly              | Batch API is fine                                 |
| inconsistent results, contradictory feedback | Split into smaller focused passes                 |
| false positives, developer trust             | Write explicit criteria                           |
| hangs, waiting for input                     | -p / --print                                      |
| every developer when they clone the repo     | Project scope: .claude/... in the repository      |
| spread throughout the codebase               | .claude/rules/ with glob patterns                 |
| dozens of files, architectural decisions     | Plan mode                                         |
| single-file fix with a clear stack trace     | Direct execution                                  |

### File paths

|                                           |                                            |
|-------------------------------------------|--------------------------------------------|
| Personal instructions - <b>NOT shared</b> | ~/claude/CLAUDE.md                         |
| Project instructions - shared             | .claude/CLAUDE.md or CLAUDE.md in the root |
| Folder instructions                       | CLAUDE.md inside that folder               |
| Rule files by topic                       | .claude/rules/                             |
| Shared slash commands                     | .claude/commands/                          |
| Personal slash commands                   | ~/claude/commands/                         |
| Shared skills                             | .claude/skills/                            |
| Personal skills                           | ~/claude/skills/                           |
| Shared MCP servers                        | .mcp.json                                  |
| Personal MCP servers                      | ~/claude.json                              |

Rule: anything starting with ~/ is personal and is NOT shared through version control.

### Frontmatter keys - do not mix these two

| SKILL.md (in .claude/skills/)                                                                    | .claude/rules/ files                                                           |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>context:</b> fork run in a separate context, keeping long output out of the main conversation | <b>paths:</b> glob patterns. The rule loads only when you edit a matching file |
| <b>allowed-tools</b> limit which tools the skill may use                                         | Example:<br>paths: ["terraform/**/*"]<br>paths: ["**/*.test.tsx"]              |
| <b>argument-hint</b> ask the developer for a missing parameter                                   |                                                                                |

**paths: is NOT a SKILL.md key.** Mixing these two sets is the most common mistake in Domain 3.

### Commands and flags

|                                             |                      |
|---------------------------------------------|----------------------|
| See which memory files are loaded           | /memory              |
| Reduce context in a long session            | /compact             |
| Continue a named session                    | --resume <name>      |
| Branch from one shared analysis             | fork_session         |
| Non-interactive mode in CI - fixes hanging  | -p or --print        |
| Output a script can read                    | --output-format json |
| Force output to follow a schema             | --json-schema        |
| Noisy discovery without filling the context | Explore subagent     |

| These DO NOT EXIST - they appear as wrong answers          |
|------------------------------------------------------------|
| --batch CLAUDE_HEADLESS=true --no-interactive              |
| .claude/config.json with a commands list                   |
| Redirecting stdin from /dev/null as the "correct approach" |

### 3 - Domain 1: Agentic Architecture (27%)

The biggest domain. About 16 of the 60 questions.

#### The agentic loop

|                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The loop is controlled by <b>stop_reason</b> . Nothing else.                                                                                                                                                                                                                                                                                                                                                                 |
| <b>"tool_use"</b> -> run the tools, add results to history, repeat                                                                                                                                                                                                                                                                                                                                                           |
| <b>"end_turn"</b> -> stop                                                                                                                                                                                                                                                                                                                                                                                                    |
| All seven API values: end_turn, max_tokens, stop_sequence, tool_use, pause_turn, refusal, model_context_window_exceeded. The exam tests the first pair                                                                                                                                                                                                                                                                       |
| Every tool_use needs one tool_result with the same <b>tool_use_id</b>                                                                                                                                                                                                                                                                                                                                                        |
| Send all results from one Claude message back in <b>ONE</b> user message                                                                                                                                                                                                                                                                                                                                                     |
| <b>Four wrong ways to stop the loop</b> <ol style="list-style-type: none"> <li>1. Reading Claude's text for a phrase like "I have completed"</li> <li>2. A loop counter as the MAIN stopping method (as a safety limit it is fine)</li> <li>3. Checking whether the response contains text - text and tool calls appear together</li> <li>4. An explicit task_complete tool - end_turn already signals completion</li> </ol> |

#### Subagents

|                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Started with the <b>Task tool</b>                                                                                                                                                                                           |
| The coordinator's <b>allowedTools must include "Task"</b> - without it, no delegation is possible                                                                                                                           |
| Subagents receive <b>NOTHING</b> automatically. No history, no memory. Put it all in the prompt                                                                                                                             |
| <b>Parallel</b> = several Task calls in <b>ONE response</b>                                                                                                                                                                 |
| <b>fork_session</b> = branches from ONE shared analysis, to compare approaches                                                                                                                                              |
| All messages go <b>through the coordinator</b> . Subagents never talk to each other                                                                                                                                         |
| <b>AgentDefinition</b> = each subagent's description, system prompt, and tool restrictions                                                                                                                                  |
| <b>The failure to remember</b> <p>Coverage is incomplete BUT every subagent worked correctly</p> <p>= the coordinator's <b>task decomposition was too narrow</b></p> <p>Any answer blaming a downstream agent is wrong.</p> |

#### Making a rule that always works

| Programmatic enforcement                          | Prompt-based guidance            |
|---------------------------------------------------|----------------------------------|
| Hooks, prerequisite gates                         | System prompt, few-shot examples |
| Always works                                      | Fails sometimes                  |
| Money, identity checks, policy limits, compliance | Style, preferences, judgement    |

#### The two hook types

|                                              |                                                                                                                                               |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PostToolUse</b>                           | Changes tool <b>RESULTS</b> before the model reads them.<br>Use for normalising different data formats (Unix time / ISO 8601 / numeric codes) |
| <b>PreToolUse</b><br>(the interception hook) | Blocks an outgoing tool <b>CALL</b> .<br>Use for blocking refunds over \$500 and redirecting to escalation                                    |

#### Splitting up work

Ask first: **do I know the shape of the work before I start?**

|                                                    |                                                                                                                                  |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Prompt chaining</b><br>YES - shape known        | The file list is known and the criteria are fixed.<br>Large review = <b>one pass per file + one pass across files</b>            |
| <b>Dynamic decomposition</b><br>NO - shape unknown | Structure unknown; each step depends on the last.<br>Map first, then a plan that adapts. Example: add tests to a legacy codebase |

"Audit", "large" and "comprehensive" do **NOT** decide this. 900 files, nobody knows where the code lives = **dynamic**. 22 files, known list = **chaining**. Same size, opposite answers.

A bigger context window does **NOT** fix attention dilution.

#### Sessions

|                                              |                                                   |
|----------------------------------------------|---------------------------------------------------|
| A few known files changed                    | --resume, and name the changed files              |
| Earlier tool results are <b>mostly stale</b> | Start a <b>NEW</b> session with a written summary |

The question is **HOW MUCH** is stale, not whether anything is stale.

## 4 - Domain 2: Tool Design and MCP (18%)

About 11 of the 60 questions.

### Tool descriptions - the core fact

|                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------|
| Tool <b>descriptions</b> are how the model chooses a tool. Improve them <b>first</b> .                                                         |
| A description must contain: <b>inputs</b> , <b>example queries</b> , <b>edge cases</b> , and <b>when to use it instead of a similar tool</b> . |
| Overlapping tools -> <b>rename</b> and rewrite (analyze_content -> extract_web_results)                                                        |
| One tool doing three jobs -> <b>split</b> it, with defined inputs and outputs                                                                  |
| Check the <b>system prompt</b> for words that create unwanted tool links                                                                       |
| Improve MCP descriptions, or the agent prefers built-in Grep over a better MCP tool                                                            |

### Error responses

| Category          | Example               | Agent should                      |
|-------------------|-----------------------|-----------------------------------|
| <b>transient</b>  | Timeout, service down | Retry                             |
| <b>validation</b> | Bad input             | Fix the input, retry              |
| <b>business</b>   | Breaks a policy       | Explain to the user. Do NOT retry |
| <b>permission</b> | Not allowed           | Escalate or explain               |

|                                                                                                         |
|---------------------------------------------------------------------------------------------------------|
| Every error returns: <b>errorCategory</b> + <b>isRetryable</b> + a readable message                     |
| Business rules also need <b>isRetryable: false</b> and a customer-friendly explanation                  |
| An <b>access failure is NOT an empty result</b> . A timeout may need a retry; zero matches is a success |
| Subagents fix transient failures <b>themselves</b> . They only report what they cannot fix              |
| When reporting: failure type + what was tried + partial results + alternatives                          |

### tool\_choice

|                                   |                                                                                          |
|-----------------------------------|------------------------------------------------------------------------------------------|
| <b>auto</b>                       | May reply with <b>TEXT</b> instead of calling a tool. Never the fix when it returns text |
| <b>any</b>                        | Must call a tool, chooses which. Use when the document type is unknown                   |
| <b>{"type":"tool","name":...}</b> | Must call THAT tool. Use to force extract_metadata to run first                          |
| <b>none</b>                       | Cannot use tools                                                                         |

auto may talk - any must act - forced picks the actor.

### Tool distribution

|                                                                                      |
|--------------------------------------------------------------------------------------|
| <b>18 tools is bad. 4-5 is good.</b> More tools = worse selection                    |
| An agent with tools outside its job <b>will misuse them</b>                          |
| Give a small cross-role tool for a common need (verify_fact for the synthesis agent) |
| Replace general tools with limited ones (fetch_url -> load_document)                 |

### MCP configuration

|                              |                                                                                                              |
|------------------------------|--------------------------------------------------------------------------------------------------------------|
| Shared team servers          | .mcp.json (project, version-controlled)                                                                      |
| Personal / experimental      | ~/.claude.json (user)                                                                                        |
| Keep secrets out of the repo | \${GITHUB_TOKEN} environment variable expansion                                                              |
| <b>MCP resources</b>         | Content catalogues: issue lists, doc structures, DB schemas.<br>They stop the agent making exploratory calls |
| Standard integration (Jira)  | Use a <b>community</b> server. Build custom only for your own workflows                                      |

### Built-in tools

|                                   |                                                                                |
|-----------------------------------|--------------------------------------------------------------------------------|
| <b>Grep</b>                       | Searches INSIDE files - function callers, error messages, imports              |
| <b>Glob</b>                       | Matches FILE PATHS - **/*.test.tsx                                             |
| <b>Edit</b> fails on unclear text | Use <b>Read + Write</b> instead                                                |
| Exploring a codebase              | Grep to find entry points, then Read to follow imports. Do not read everything |

## 5 - Domain 4: Prompts and Structured Output (20%)

Your weakest domain on the day-1 test. Read this page every day.

### Schema design

|                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Required or nullable? Ask: does the source ALWAYS contain this value?</b><br>Source may NOT contain it → <b>nullable</b> , or a required field makes the model <b>INVENT</b> one.<br>Source ALWAYS contains it → <b>required</b> , so the field cannot be <b>OMITTED</b> .<br>Never "nullable + reject null" - that fails documents and still does not stop fabrication |
| Enum + <b>"other"</b> + a detail string, for categories that may grow                                                                                                                                                                                                                                                                                                      |
| Enum value <b>"unclear"</b> for ambiguous cases                                                                                                                                                                                                                                                                                                                            |
| Tool use + schema removes <b>SYNTAX</b> errors. It does NOT remove <b>SEMANTIC</b> errors                                                                                                                                                                                                                                                                                  |
| <b>calculated_total</b> next to <b>stated_total</b> - catches totals that do not add up                                                                                                                                                                                                                                                                                    |
| <b>conflict_detected</b> - boolean for contradictory source data                                                                                                                                                                                                                                                                                                           |
| <b>detected_pattern</b> - to analyse which findings developers dismiss                                                                                                                                                                                                                                                                                                     |
| Put format normalisation rules in the <b>prompt</b> , next to the strict schema                                                                                                                                                                                                                                                                                            |

### Retries

| Retry WORKS                    | Retry DOES NOT WORK                           |
|--------------------------------|-----------------------------------------------|
| Wrong format (dates, currency) | The information is <b>not in the document</b> |
| Wrong output structure         | It is only in a document you did not supply   |

A retry request contains: the **document** + the **failed extraction** + the **specific validation errors**.

### Message Batches API

|                         |                                                                             |
|-------------------------|-----------------------------------------------------------------------------|
| Cost                    | <b>50% cheaper</b>                                                          |
| Time                    | <b>Up to 24 hours</b>                                                       |
| Speed guarantee         | <b>NONE</b>                                                                 |
| Matching results        | <b>custom_id</b> (never by position)                                        |
| Multi-turn tool calling | <b>Not supported</b>                                                        |
| Good for                | Overnight reports, weekly audits, nightly test generation                   |
| Bad for                 | Anything where a person is waiting (pre-merge, pre-commit)                  |
| Failures                | Resubmit <b>only</b> the failures, found by custom_id. Split long documents |
| SLA maths               | 24-hour window + 30-hour SLA -> submit every <b>4 hours</b>                 |

### Prompts and criteria

|                                                                                                         |
|---------------------------------------------------------------------------------------------------------|
| <b>Specific categories</b> work. <b>"Be conservative"</b> does not                                      |
| Confidence-based filtering does <b>not</b> improve precision                                            |
| A category with many false alarms damages trust in the good ones -> <b>turn it off</b> while you fix it |
| Severity levels need <b>concrete code examples</b> for each level                                       |
| <b>Few-shot: 2 to 4 examples</b> , aimed at the <b>unclear</b> cases, not the obvious ones              |
| Few-shot lets the model <b>generalise</b> to new situations, not just copy                              |

### Review architecture

|                                                                                                  |
|--------------------------------------------------------------------------------------------------|
| Self-review is weak: the model remembers its own reasoning -> use an <b>independent instance</b> |
| Large review = <b>one pass per file + one separate pass across files</b>                         |
| A bigger context window does <b>NOT</b> fix attention dilution                                   |
| Voting across runs <b>hides real bugs</b>                                                        |
| Confidence IS allowed next to each finding, to route reviewer attention                          |

## 6 - Domain 5: Context and Reliability (15%)

Only 15%, but it appears in four of the six exam scenarios.

### Three context problems and their fixes

| Problem                                                                                 | Fix                                                                           |
|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Progressive summarization</b> loses numbers, dates, and what the customer expected   | A " <b>case facts</b> " block in every prompt, OUTSIDE the summarised history |
| <b>Lost in the middle</b> - the model handles the start and end well, misses the middle | Key findings at the <b>BEGINNING</b> . Clear section headings for the details |
| <b>Tool results fill the context</b> - 40+ fields when only 5 matter                    | <b>Trim to the useful fields BEFORE</b> they enter the context                |

### Escalation

| Escalate when                                       | NEVER escalate based on                           |
|-----------------------------------------------------|---------------------------------------------------|
| The customer <b>asks</b> for a human                | <b>Sentiment</b> - it does not predict difficulty |
| There is a <b>policy gap</b> (not just a hard case) | The model's own <b>confidence score</b>           |
| No progress is possible                             |                                                   |

#### The trap - these two look the same

Customer **explicitly asks** for a human -> escalate **IMMEDIATELY**, do not investigate first

Customer is **frustrated** but you can help -> offer help, escalate only if they ask again

Several matching customers -> **ask for another identifier**. Never guess.

### Errors between agents

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| Report four things: <b>failure type + what was tried + partial results + alternatives</b> |
| <b>Never:</b> a generic message like "search unavailable"                                 |
| <b>Never:</b> hiding the error by returning empty results as success                      |
| <b>Never:</b> stopping the whole workflow because one agent failed                        |
| <b>Never:</b> treating an access failure and a valid empty result as the same thing       |
| Synthesis output should mark which topics have <b>coverage gaps</b>                       |

### Long codebase exploration

|                                       |                                                                                            |
|---------------------------------------|--------------------------------------------------------------------------------------------|
| Symptom of <b>context degradation</b> | The model talks about " <b>typical patterns</b> " instead of the specific classes it found |
| Fix                                   | <b>Scratchpad files</b> - save key findings, read them later                               |
| Also                                  | Subagent delegation, phase summaries, /compact                                             |
| Crash recovery                        | <b>Manifests</b> - each agent exports state; the coordinator loads it on resume            |

### Human review and confidence

|                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------|
| <b>97% average accuracy can hide one broken document type</b>                                                       |
| Analyse accuracy <b>by document type and by field</b> before reducing human review                                  |
| <b>Stratified random sampling</b> of HIGH-confidence extractions - that is the group you are about to stop checking |
| Confidence is usable <b>only when calibrated with labelled validation sets</b> , and only for routing review        |
| Route to a human: low confidence, or the source is ambiguous or contradicts itself                                  |

### Sources and provenance

|                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Attribution is lost during <b>summarisation</b> , when claim-source links are not preserved                                                                                                    |
| Subagents output <b>claim-source mappings</b> : source URL, document name, relevant excerpt                                                                                                    |
| Subagents must include <b>publication dates</b> , or old and new numbers look like a contradiction                                                                                             |
| Also <b>source location</b> and <b>methodology</b> - nobody downstream can recover these.<br>Never "weigh credibility" or "re-fetch at synthesis time": provenance travels as <b>structure</b> |
| Two credible sources disagree -> <b>record BOTH with their sources</b> . Never pick one                                                                                                        |
| Separate <b>well-established</b> findings from <b>contested</b> ones in the report                                                                                                             |
| Format by content type: financial data = <b>tables</b> , news = <b>prose</b> , technical = <b>lists</b>                                                                                        |